

Hlavní Struktura Projektu

Projekt je navržený s ohledem na objektově orientované programování (OOP) a zachycení jednotlivých instrukcí IPPcode24 pomocí tříd a vzorů návrhu. Kód využívá abstrakce, dědičnosti a polymorfismu, aby zajistil modularitu a snadnou rozšiřitelnost.

Třída `AbstractInstruction`

Abstraktní třída `AbstractInstruction` funguje jako šablona pro instrukce. Definuje základní rozhraní pro všechny instrukce a zajišťuje, že každá konkrétní instrukce bude mít implementovanou metodu `execute`. Díky tomu může hlavní část interpretu spouštět instrukce bez znalosti specifiků každé z nich.

Instrukce a jejich Implementace

Každá instrukce je reprezentována samostatnou třídou, která dědí od `AbstractInstruction`. Tyto třídy převádějí logiku dané instrukce do metod a mohou být testovány nezávisle na ostatních částech kódu. Příkladem může být `MoveInstruction`, která implementuje přesun hodnot mezi proměnnými, nebo `CreateFrameInstruction`, jež spravuje vytváření nových rámců.

Rámce a Zásobníky

Srdcem interpretu jsou rámce (`Frame`) a zásobníky (`callStack` a `operandStack`), které poskytují prostředky pro práci s proměnnými a správu toku programu. Třída `Frame` má metody pro definování proměnných, získání hodnot a správu dočasných rámců. Zásobník volání uchovává informace potřebné pro návrat z funkčních volání, zatímco zásobník operandů slouží pro mezivýsledky v operacích.

Výjimky a Chybové Stavy

Pro zachycení a zpracování chybových stavů projekt definuje několik tříd výjimek. Tyto výjimky jsou vyvolány v situacích, jako je chybný typ operandu, sémantická chyba nebo chybějící hodnota. Přesné a deskriptivní výjimky pomáhají při ladění a zajištění kvality kódu.

Validace a Testování

Důkladná validace vstupních argumentů a datových typů zajišťuje robustnost interpretu. Projekt byl testován na několika příkladech, kdy bylo využito několik set testů, jak už dohromady s již odevzdaným parserem nebo i samostatných testů na interpret, které testovali nejrůznější situace, které mohou nastat.

Popis Tříd `Frame` a `Variable`

Třída `Frame` zastupuje rámec pro proměnné ve v interpretu. Každý rámec může obsahovat více proměnných a poskytuje metody pro jejich manipulaci – přidání nové proměnné (`defineVar`), získání hodnoty proměnné (`getVariableValue`) nebo nastavení hodnoty (`setVariableValue`). To je klíčové pro správu lokálního prostředí během vykonávání programu.

Třída `Variable` reprezentuje individuální proměnné uvnitř rámce. Obsahuje informace o názvu proměnné a její aktuální hodnotě. Třída umožňuje získání a nastavení hodnoty proměnné, což je nezbytné pro operace jako přiřazení a čtení hodnot.

Popis Třídy InstructionValidator

Třída `InstructionValidator` poskytuje metody pro ověření, že XML elementy reprezentující jednotlivé instrukce `IPPCode24` obsahují přesně stanovený počet argumentů. Metody `validateZeroArgs`, `validateOneArg`, `validateTwoArgs` a `validateThreeArgs` zkontrolují, že instrukce má respektive 0, 1, 2, nebo 3 argumenty, přičemž přítomnost nadbytečných argumentů vede k vyhození výjimky `InvalidSource`. Tato třída je zásadní pro zajištění správné syntaxe a struktury kódu při jeho analýze.

Popis Třídy StringDecoder

Třída `StringDecoder` slouží k převodu escape sekvencí reprezentujících ASCII znaky v řetězcích z formátu `IPPCode24` na příslušné znaky. Statická metoda `decodeEscapeSequences` přijímá řetězec s escape sekvencemi a vrací upravený řetězec, ve kterém jsou sekvenční kódy nahrazeny odpovídajícími ASCII znaky. K převodu využívá regulární výrazy a zachází s escape sekvencemi reprezentujícími znaky s hodnotami od 0 do 99 v oktální soustavě. Tato metoda zaručuje správnou interpretaci řetězců v kódu.

Popis Třídy XMLProcessor

Třída `XMLProcessor` je zodpovědná za čtení a zpracování XML reprezentace `IPPCode24`, zajišťuje správné načtení a ověření kořenového elementu a extrahuje z něj všechny instrukce. Využívá `DOMXPath` pro efektivní práci s XML a kontroluje správné pořadí a typy instrukcí. Při zpracování každé instrukce zároveň registruje štítky v kontextovém rámci. Je nezbytná pro předzpracování kódu před jeho spuštěním interpretem a zvyšuje modularitu a rozšiřitelnost celého projektu.

UML Diagram

